

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 24: Virtual Memory

Instructor: Justin Yokota
Slide Credit: Lisa Yan

Agenda

- Virtual Memory and Virtual Addresses
- Paged Memory and Address Translations
- Practice
- Page Table Details
- Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs Virtual Memory

Agenda

- **Virtual Memory and Virtual Addresses**
- Paged Memory and Address Translations
- Practice
- Page Table Details
- Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs Virtual Memory

How much memory is my computer using?

- My computer reports that I'm using 13.8 GB of Random Access Memory (out of 15.7 GB total)
- Let's verify this by calculating from some other data.
 - Might be a bit off because of some factor we don't know yet, but it shouldn't be too far off...
- Currently my computer is using 422 processes, each of which needs an address space
- My computer is a 64-bit computer, so memory addresses are 48 bits long
- Total = $422 * 2^{48}$ bytes = 100 petabytes (about 8 million times off)
 - Wait...

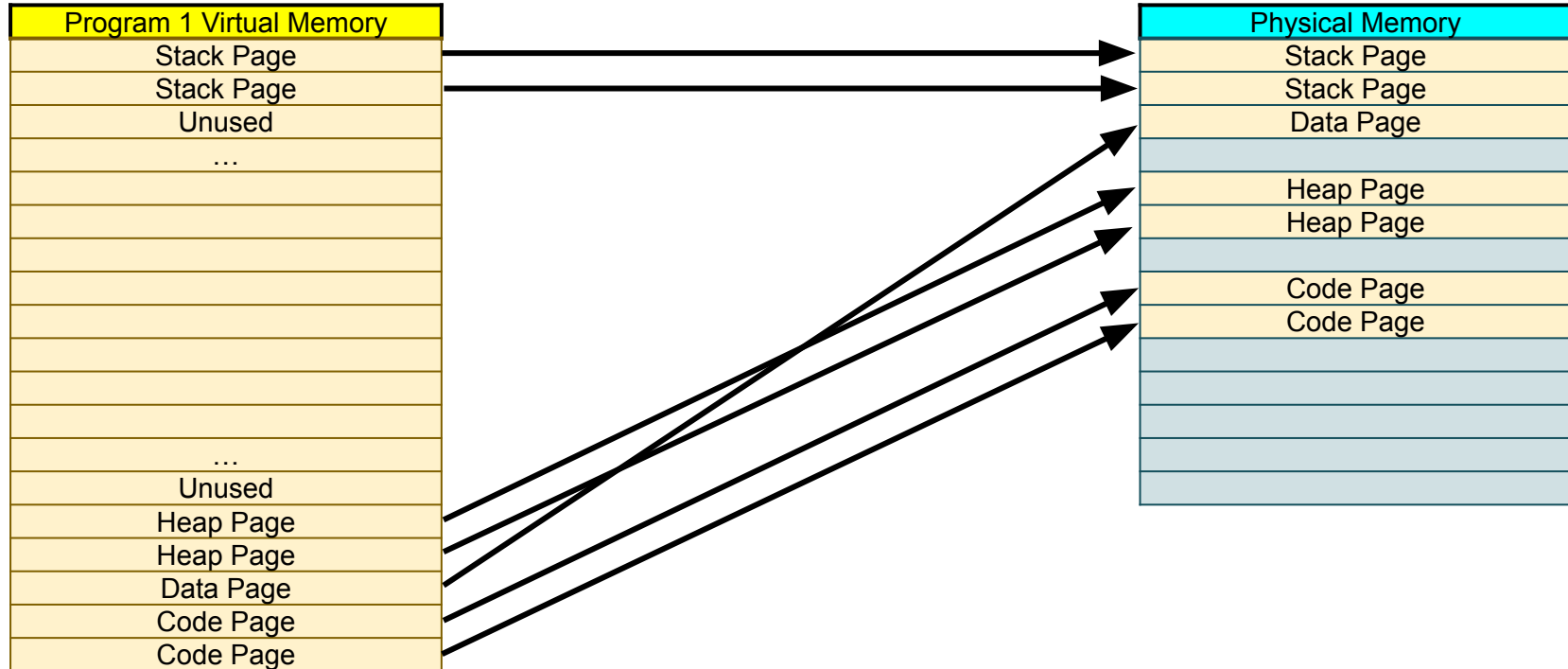
Virtual Memory

- If we gave each process a full 2^{48} bytes of RAM, we won't be able to fit any processes in our RAM.
- Can we do better?
- Most processes don't use all their address space.
 - They mostly use one block of memory in the code segment, another block in the stack, and maybe a few blocks in the heap/data segments.
- Solution: Chop up main memory into "pages" of a few KiB each. Only store pages that a program actually tries to access.
 - Ex. A program may be using 1 page for its code, 1 page for its stack, 4 pages for its heap, etc.
- When we run a program, it needs to believe that it has access to its full 2^{32} bit address space (on a 32-bit machine). This memory doesn't actually exist; it's **"virtual" memory**.
- When a program actually needs a page of memory, we'll allocate a page of **physical memory**.

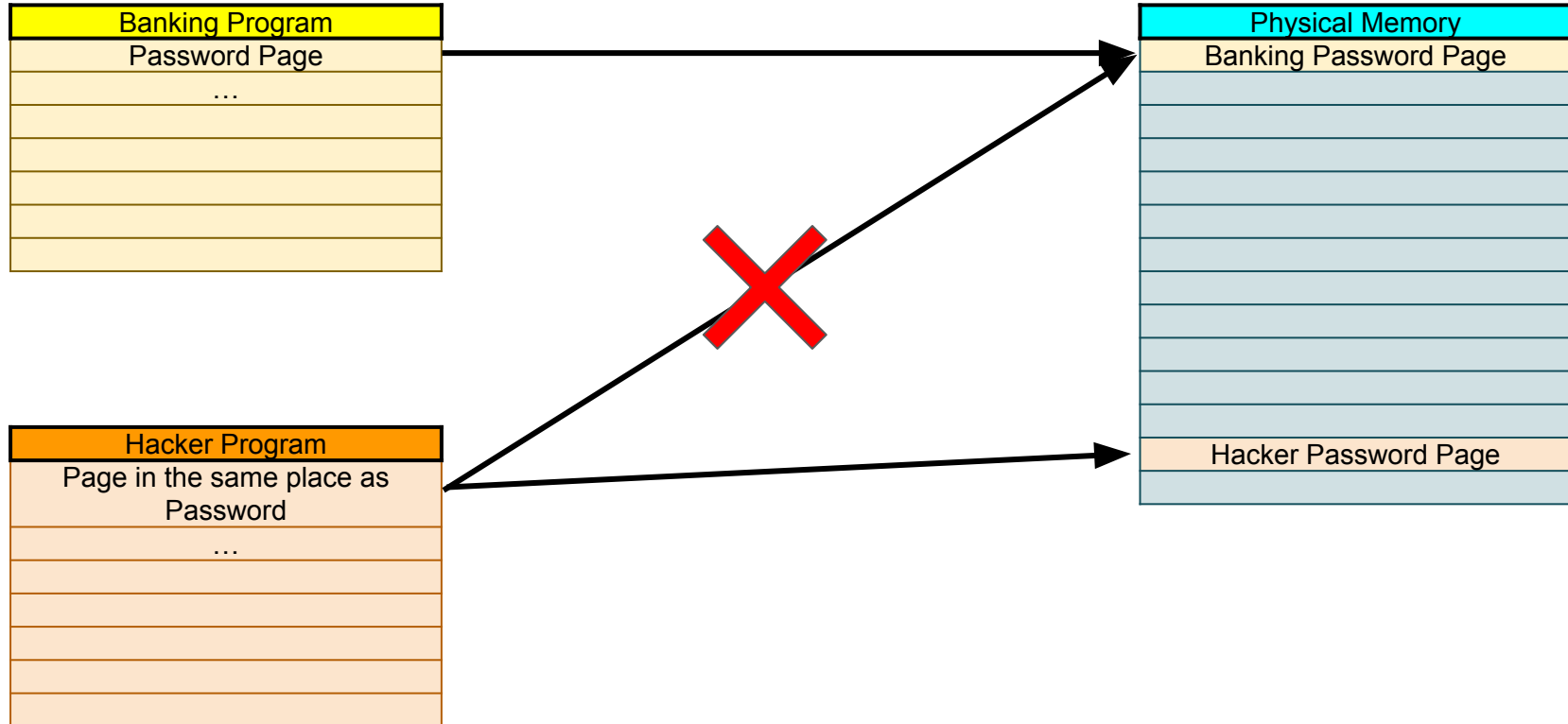
Virtual Memory

- **Virtual memory** is the next level in the memory hierarchy:
 - Give each process the **illusion** of a **full memory address space** that it has completely for itself.
 - Under the hood: working set of **pages** reside in main memory; other pages are on **disk**.
- **Benefits:**
 - **Demand paging** provides the ability to run programs larger than the primary memory (DRAM).
 - OS can share memory and **protect** programs from each other.
 - Hides differences between machine configurations.
 - (Historically, virtual memory predates caches.)
- Logic here wants more flexibility than hardware can permit. So we'll incorporate it in the OS.

Virtual Memory Saves Unused Pages

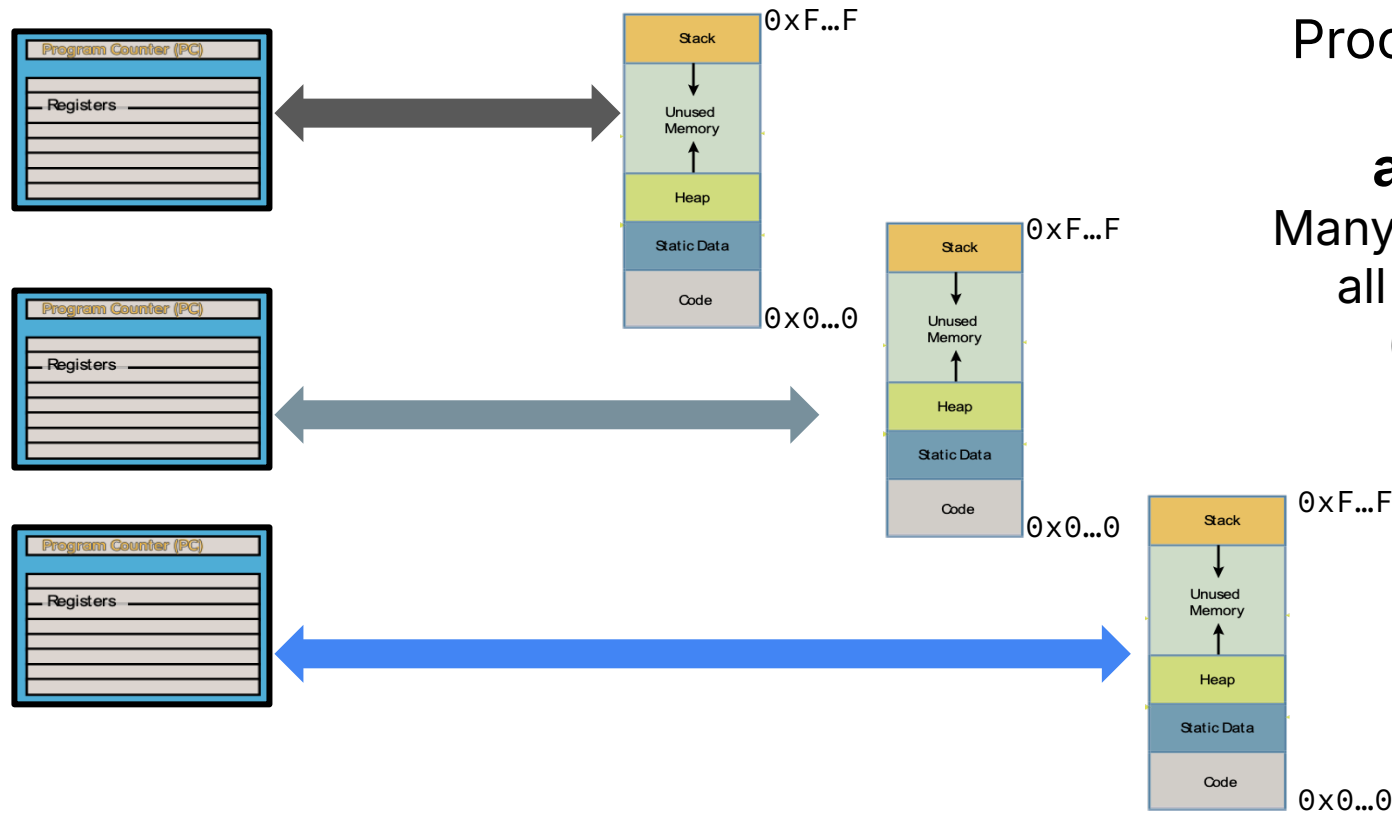


Virtual Memory Protects Programs from Each Other



The Illusion of Virtual Address Space

Different processes
run simultaneously

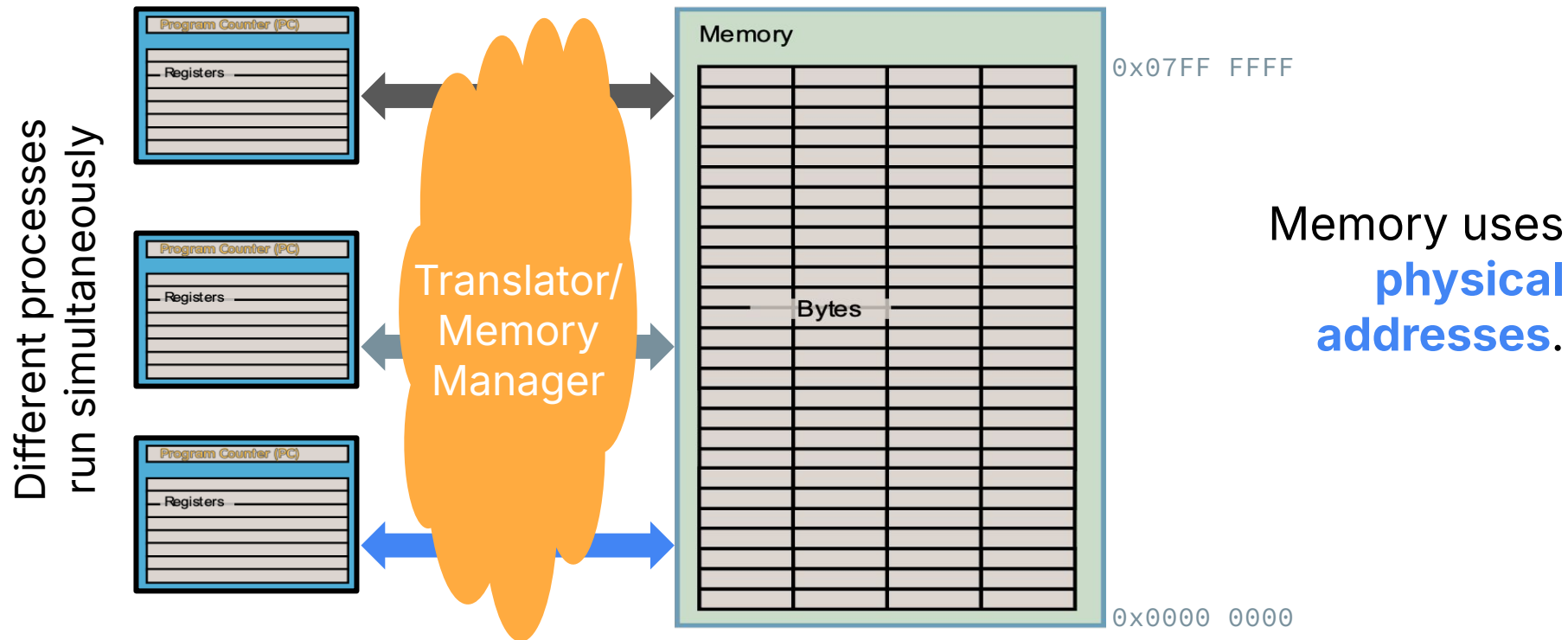


Processes use
**virtual
addresses.**
Many processes,
all using same
(conflicting)
addresses

The Translation to Virtual Address Space

CS 61C

Summer 2026



Address Sizes: The Hive Machines

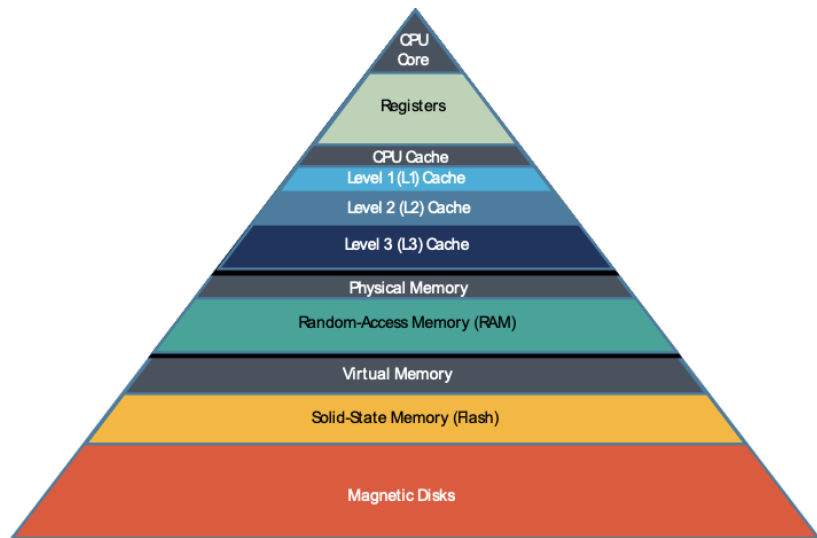
```
(00:18:45 Mon Nov 07 2022 cs61c-tab@hive2 Linux x86_64)
~ $ cat /proc/cpuinfo
processor       : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 60
model name    : Intel(R) Core(TM) i7-4770 CPU @ 3.40GHz
stepping      : 3
microcode     : 0x28
cpu MHz       : 3693.327
cache size    : 8192 KB
physical id   : 0
siblings      : 8
core id       : 0
cpu cores     : 4
apicid        : 0
initial apicid : 0
fpu           : yes
fpu_exception : yes
cpuid level   : 13
wp            : yes
flags         : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov
pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdt
scp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cp
uid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fm
a cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes x
save avx f16c rdrand lahf_lm abm cpuid_fault epb invpcid_single pti ssbd ibrs ib
pb stibp tpr_shadow vnmi flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 a
vx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln pts md_clear flush_l1d
bugs          : cpu_meltdown spectre_v1 spectre_v2 spec_store_bypass l1tf mds
swapgs itlb_multihit srbds
bogomips      : 6784.38
clflush size  : 64
cache_alignm  : 64
address sizes  : 39 bits physical, 48 bits virtual
power managem
```

Virtual Addresses vs. Physical Addresses

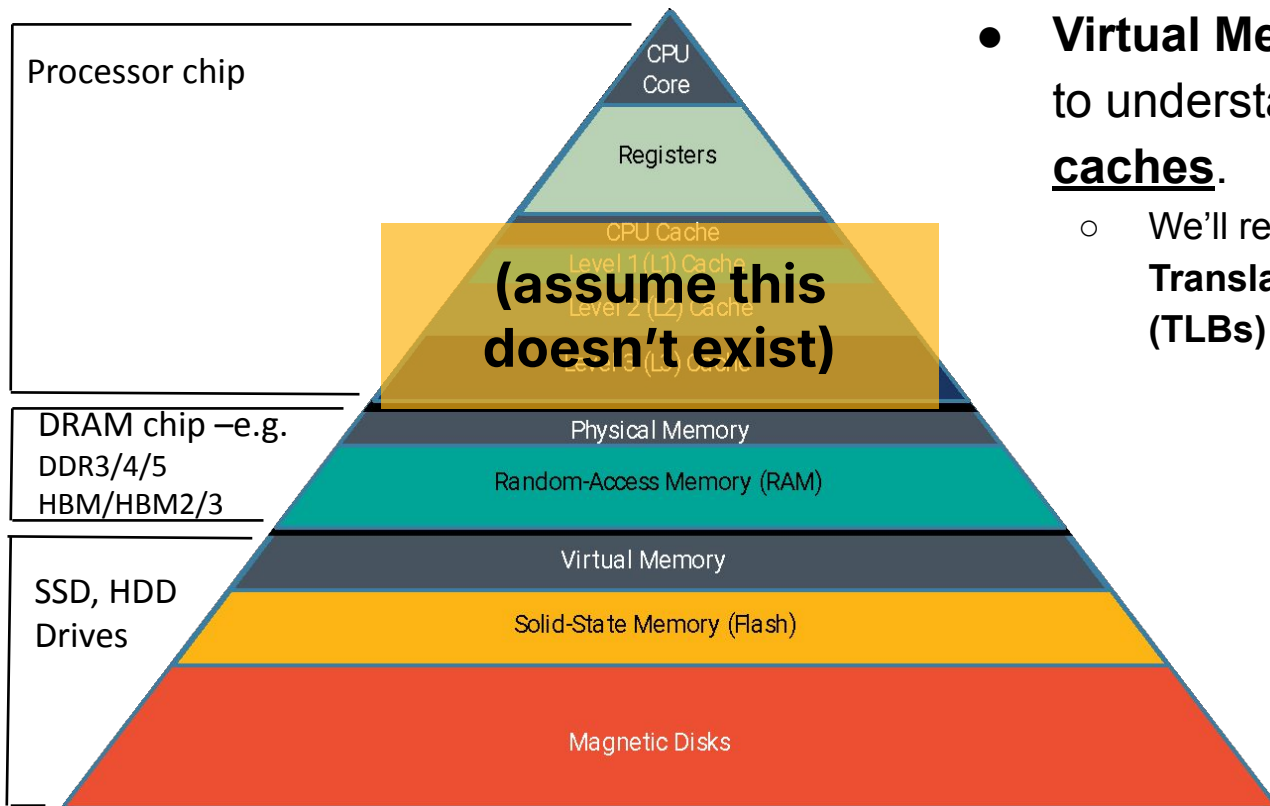
- **Address Space:** set of addresses for all available memory locations.
 - Now, two kinds of memory addresses!
- **Virtual Address Space**
 - Set of addresses that the user program knows about
- **Physical Address Space**
 - Set of addresses that map to actual physical locations in memory
 - Hidden from user applications
- For each process, the memory manager maps (**translates**) between these two address spaces.

How is the Memory Hierarchy Managed?

- registers $\leftrightarrow$ memory
 - By compiler (or assembly level programmer)
- cache $\leftrightarrow$ main memory
 - By the cache controller hardware
- main memory $\leftrightarrow$ disks (secondary storage)
 - By the operating system (virtual memory)
 - Virtual to physical address mapping assisted by the hardware ('translation lookaside buffer' or TLB)
 - By the programmer (files)



For Today: Assume Caches Don't Exist



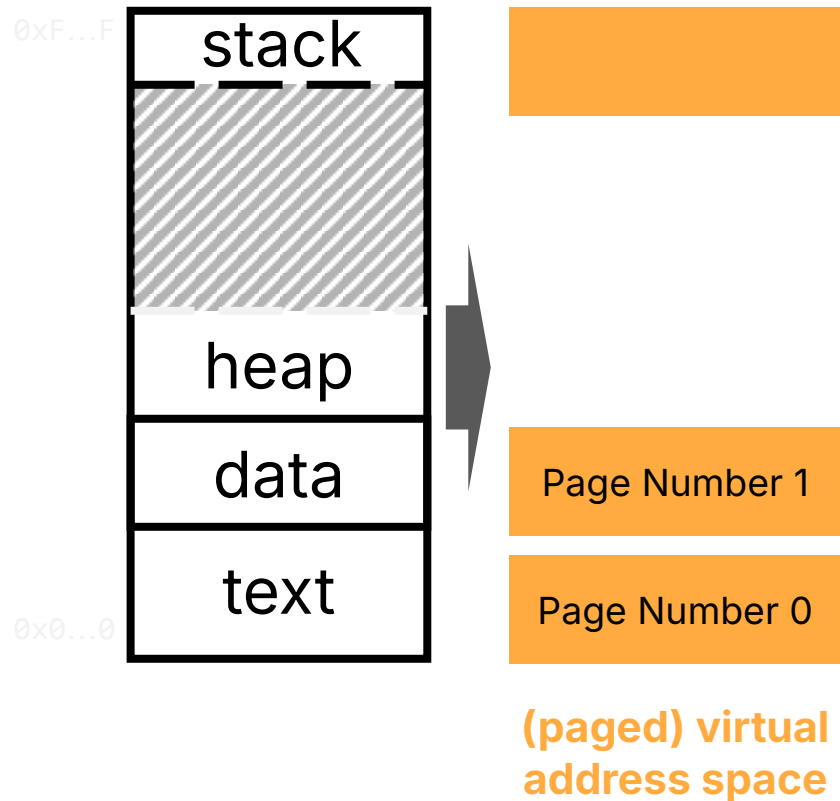
- **Virtual Memory** is much easier to understand if we assume **no caches**.
 - We'll reintroduce caches along with **Translation Lookaside Buffers (TLBs)** later.

Agenda

- Virtual Memory and Virtual Addresses
- **Paged Memory and Address Translations**
- Practice
- Page Table Details
- Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs Virtual Memory

Paged Memory

- Physical memory (DRAM) is broken into **pages**.
- A disk access loads an entire page into memory.
 - Should be large enough to amortize high access time.
- Typical page size: **4 KiB+** (on modern OSes)
 - Need 12 bits of **page offset** to address all 4 KiB bytes.



Paged Memory

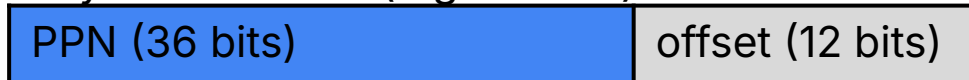
- Physical memory (DRAM) is broken into **pages**.
- A disk access loads an entire page into memory.
 - Should be large enough to amortize high access time.
- Typical page size: **4 KiB+** (on modern OSes)
 - Need 12 bits of **page offset** to address all 4 KiB bytes.

- Memory translation maps a **Virtual Page Number (VPN)** to a **Physical Page Number (PPN)**.

Virtual address (e.g. 32 bits)



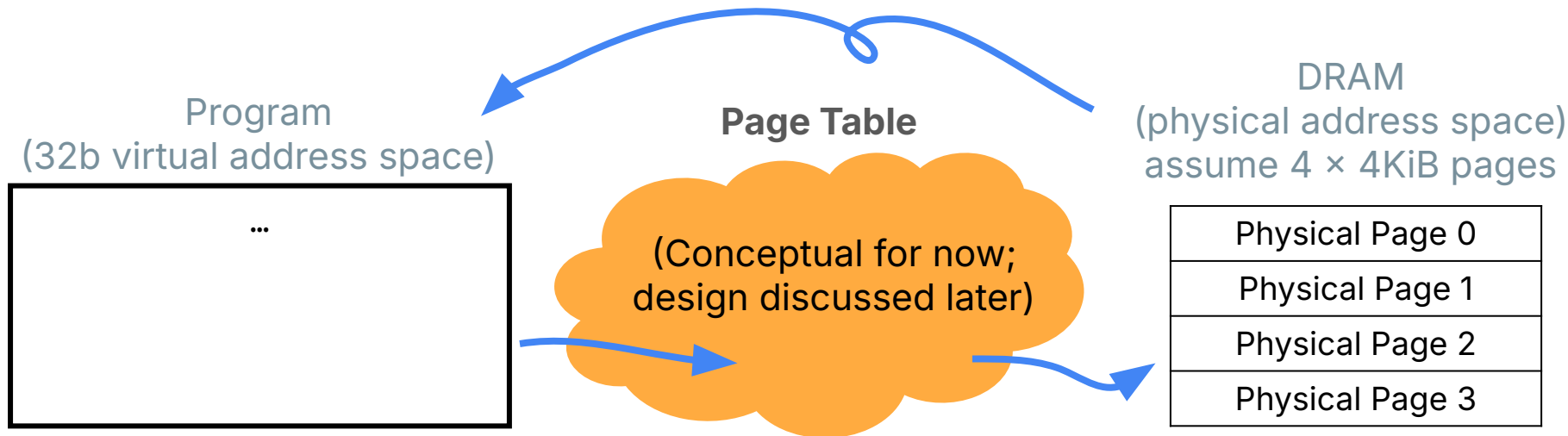
Physical address (e.g. 48 bits)



Virtual and physical pages are **the same size**. Same # bits to address offset within a page!

Address Translation

- A **page table** translates virtual addresses to physical addresses for a given process.
Each entry in the table:
 - Corresponds to a virtual page number.
 - If page is in memory, entry has corresponding physical page number.
 - Else, entry should tell OS to trigger **page fault** to **load page from disk**.



Analogy: Return of the Librarian

- Earlier, we imagined that memory was a library. For a program to access a memory address:
 - Call the librarian for the data
 - The librarian goes to that data location and responds with the data there.
- With virtual memory:
 - The librarian doesn't actually have enough books to store everyone's data.
 - So when you first start using some chunk of memory, the librarian takes a spare book and assigns it to you.
 - The librarian is responsible for translating from your memory address to the right book
 - To keep track of this, the librarian reserves one of the books for themselves and writes down each book that's being used, and who it belongs to.

Address Translation Example 1

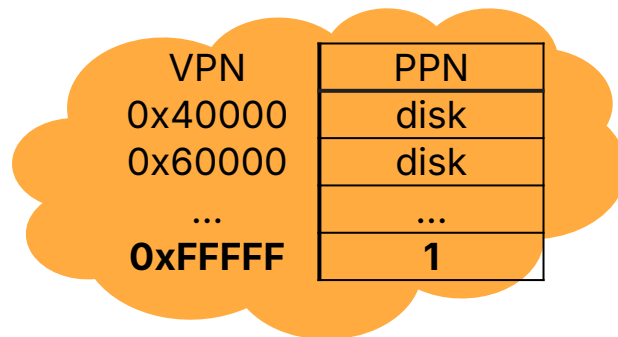
Assume each page is 4 KiB and we have 4 physical pages

Step 1: Program tries load data at 0xFFFF F004.

Program
(32b virtual address space)

Load byte @ **0xFFFF F004**
to register t0

Page Table



VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Address Translation Example 1

Step 2: OS translates **VA** to the **physical address (PA)** in memory.

- Split Virtual Address into virtual page number and offset (0xFFFF 004)
- Extract virtual page number (VPN) from VA (0xFFFF)
- Look up VPN in **page table**

Program
(32b virtual address space)

Load byte @ **0xFFFF F004**
to register t0

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFF	1

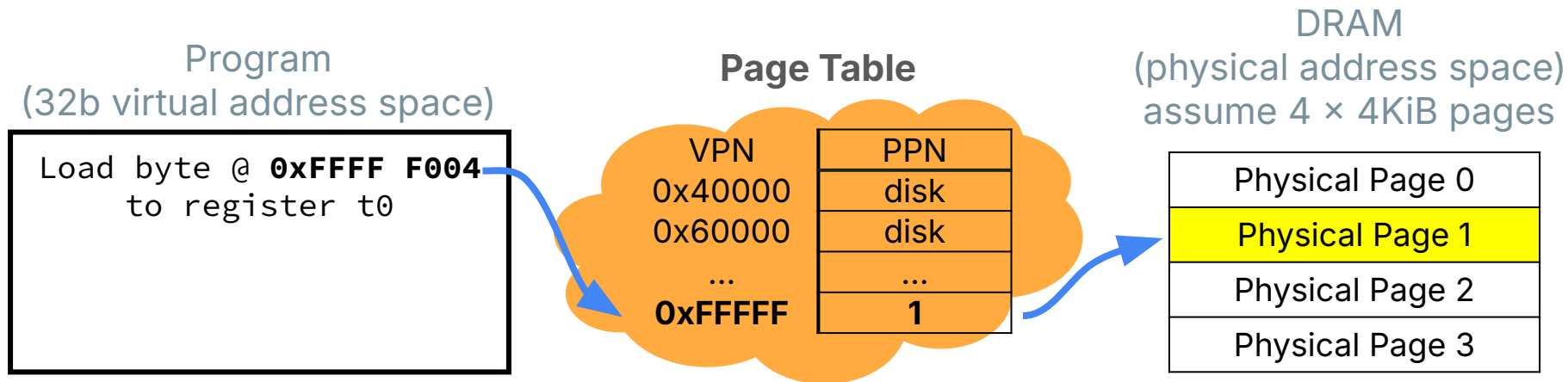
DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Address Translation Example 1

Step 3: Page table lists VPN as corresponding to a physical page. Page hit.

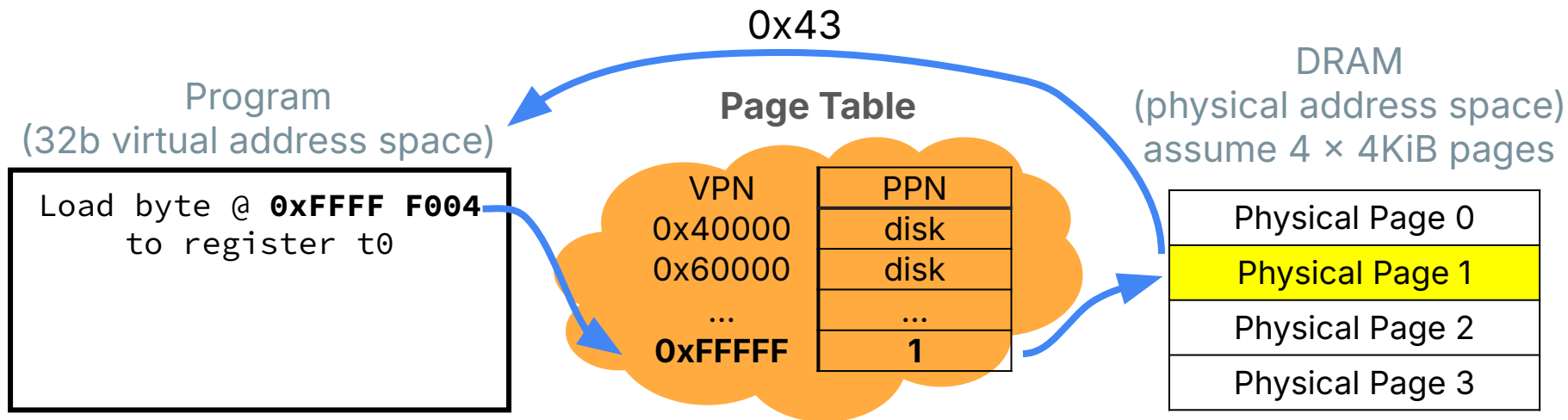
- Construct PA: physical page number + offset (from virtual address): 0x1004



Address Translation Example 1

Step 4: Read data at the corresponding memory location, and return it to the program

- Read data at 0x1004 in physical memory
- Send that data back to the program



Address Translation Example 2

Assume each page is 4 KiB and we have 4 physical pages

Step 1: Program tries to malloc new location. C library translates this to an *sbrk*, which requests an extra page of memory from the OS

Program
(32b virtual address space)

```
Request malloc(10)
sbrk requests page at
VPN 0x60000
```

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	1

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Address Translation Example 2

Step 2: OS finds empty physical page

- If no page is available, copy (evict) a page to disk and reuse that page
- OS can choose to decline the new memory allocation, in which case malloc returns NULL

Program
(32b virtual address space)

```
Request malloc(10)
sbrk requests page at
VPN 0x60000
```

Page Table

VPN	PPN
0x40000	disk
0x60000	disk
...	...
0xFFFFF	disk

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Address Translation Example 2

Step 3: OS updates Page Table, and returns the new location

Program
(32b virtual address space)

Request malloc(10)
sbrk requests page at
VPN 0x60000

Page Table

VPN	PPN
0x40000	disk
0x60000	1
...	...
0xFFFFF	disk

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Address Translation Example 3

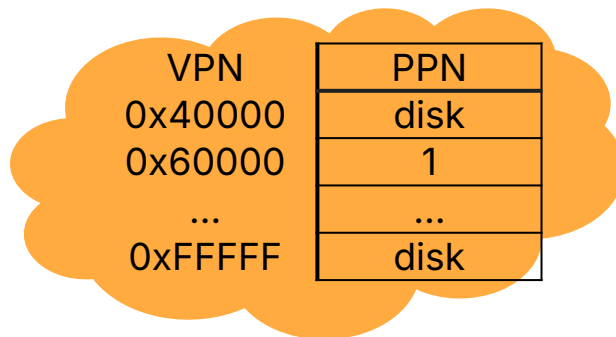
Let's try accessing the address that was just sent to disk

Step 1: Program tries load data at 0xFFFF F004.

Program
(32b virtual address space)

Load byte @ **0xFFFF F004**
to register t0

Page Table



VPN	PPN
0x40000	disk
0x60000	1
...	...
0xFFFFF	disk

DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Address Translation Example 3

Step 2: OS translates **VA** to the **physical address (PA)** in memory.

- Split Virtual Address into virtual page number and offset (0xFFFF 004)
- Extract virtual page number (VPN) from VA (0xFFFF)
- Look up VPN in **page table**

Program
(32b virtual address space)

Load byte @ **0xFFFF F004**
to register t0

Page Table

VPN	PPN
0x40000	disk
0x60000	1
...	...
0xFFFF	disk

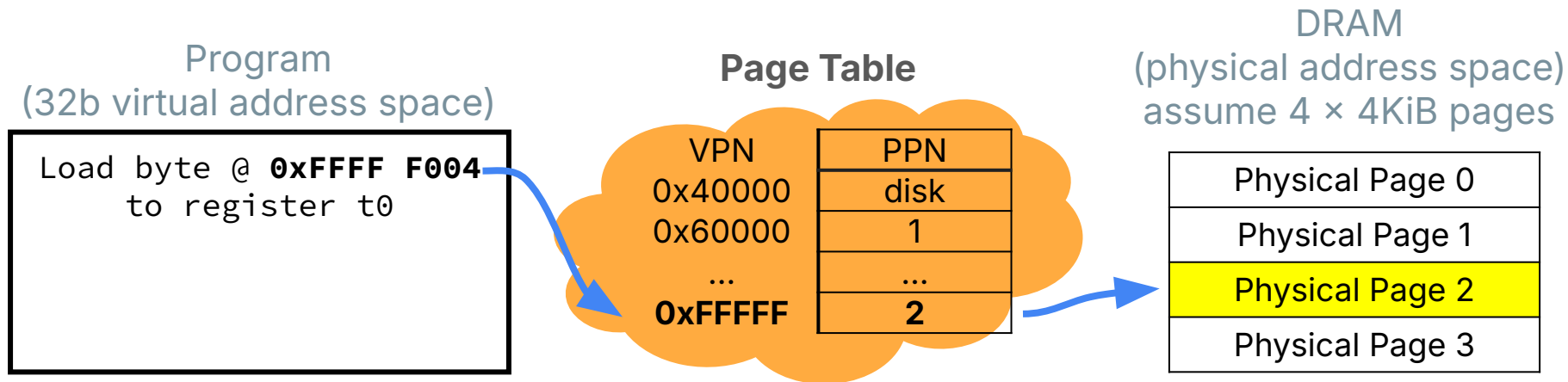
DRAM
(physical address space)
assume 4 × 4KiB pages

Physical Page 0
Physical Page 1
Physical Page 2
Physical Page 3

Address Translation Example 3

Step 3: Page table doesn't have the PPN listed. **Page Fault**

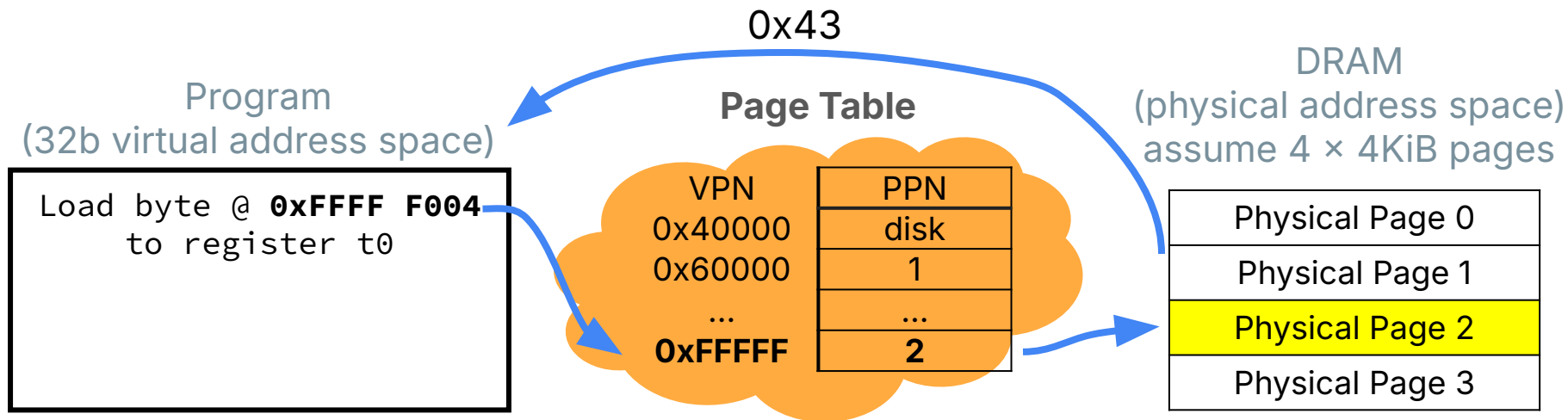
- Another part of the OS takes over here to decide if the data is on disk, or if the access is illegal
- In this case, the data is on disk, so the OS loads the data from disk back to a physical page



Address Translation Example 3

Step 4: Read data at the corresponding memory location, and return it to the program

- Read data at 0x2004 in physical memory
- Send that data back to the program



Agenda

- Virtual Memory and Virtual Addresses
- Paged Memory and Address Translations
- **Practice**
- Page Table Details
- Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs Virtual Memory

Translation Example

What Physical Address does this Virtual Address translate to?

0x00003450

- A. 0x00003450
- B. 0x0000250
- C. 0x00503450
- D. 0x0F543450
- E. 0x0F54450
- F. Disk/Other



VPN	PPN
0x00000	Disk
0x00001	0x0003
0x00002	0x0050
0x00003	0x0F54
...	
0xFFFFF	0x00F6



Translation Example: Solution

What Physical Address does this Virtual Address translate to?

0x00003450

- A. 0x00003450
- B. 0x0000250
- C. 0x00503450
- D. 0x0F543450
- E. 0x0F54450
- F. Disk/Other



VPN	PPN
0x00000	Disk
0x00001	0x0003
0x00002	0x0050
0x00003	0x0F54
...	
0xFFFFF	0x00F6

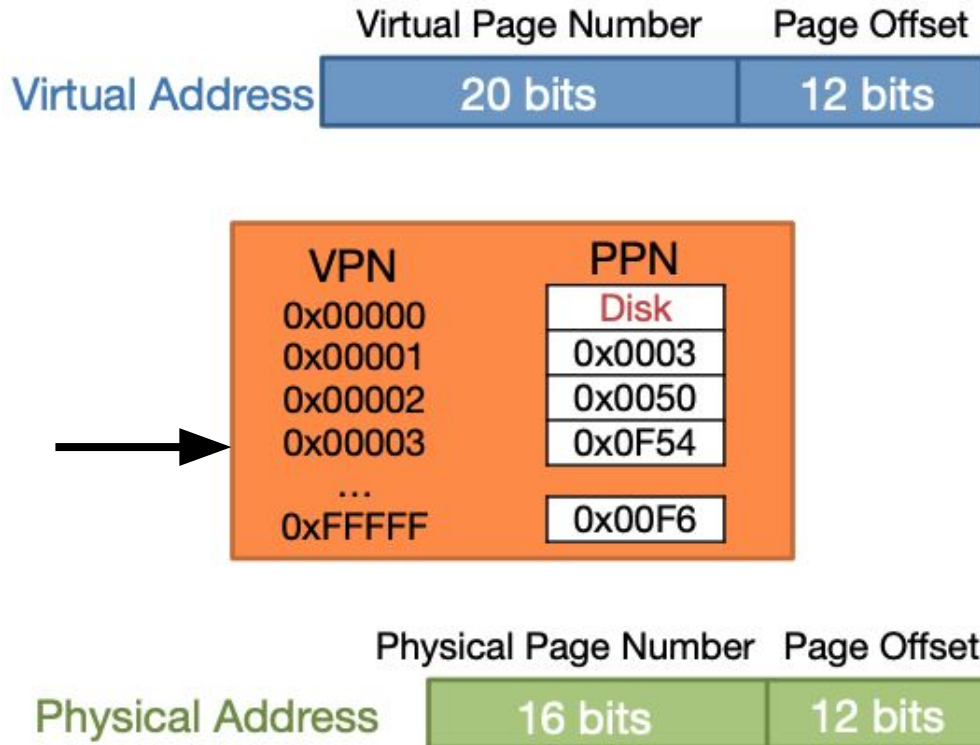


Translation Example: Solution

What Physical Address does this Virtual Address translate to?

0x00003450

- A. 0x00003450
- B. 0x0000250
- C. 0x00503450
- D. 0x0F543450
- E. 0x0F54450
- F. Disk/Other

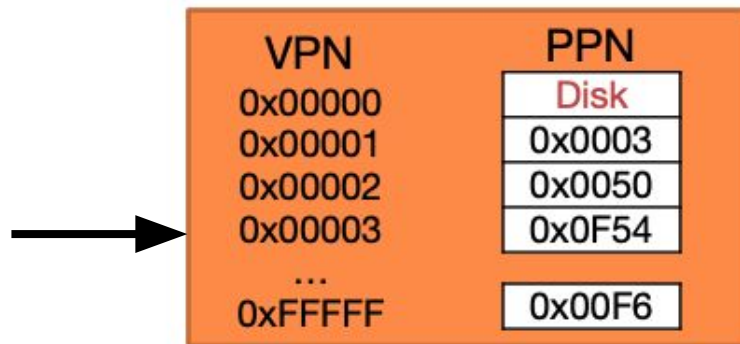


Translation Example: Solution

What Physical Address does this Virtual Address translate to?

0x00003450

- A. 0x00003450
- B. 0x0000250
- C. 0x00503450
- D. 0x0F543450
- E. **0x0F54450**
- F. Disk/Other



Determining VPN/PPN size

- Assume a 32-bit machine with 8GiB of RAM and 16KiB pages.
- How many bits would there be for each of the following?

Virtual address



Physical address



	A.	B.	C.	D.	E.	F.
1. Page offset	14	15	16	19	20	Other
2. VPN	14	15	16	19	20	Other
3. PPN	14	15	16	19	20	Other

Determining VPN/PPN size

- Assume a 32-bit machine with 8GiB of RAM and 16KiB pages.
- $16 \text{ KiB} = 2^{14} \rightarrow 14 \text{ bits offset}$

Virtual address



Physical address



	A.	B.	C.	D.	E.	F.
1. Page offset	14	15	16	19	20	Other
2. VPN	14	15	16	19	20	Other
3. PPN	14	15	16	19	20	Other

Determining VPN/PPN size

- Assume a 32-bit machine with 8GiB of RAM and 16KiB pages.
- 32-bit machine = 32-bit virtual addresses $\rightarrow 32 - 14 = 18$ bit VPN.

Virtual address



Physical address



	A.	B.	C.	D.	E.	F.
1. Page offset	14	15	16	19	20	Other
2. VPN	14	15	16	19	20	Other
3. PPN	14	15	16	19	20	Other

Determining VPN/PPN size

- Assume a 32-bit machine with 8GiB of RAM and 16KiB pages.
- 2^{33} bytes of RAM = 2^{33-14} pages
= 2^{19} pages $\rightarrow$ 19 bit PPN

Virtual address



Physical address



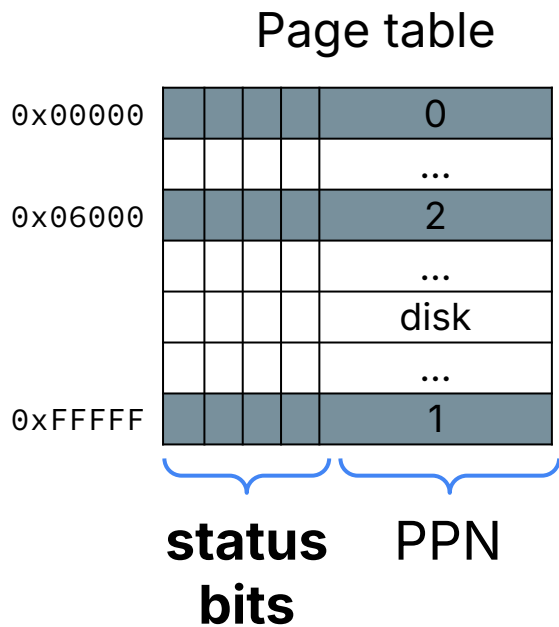
	A.	B.	C.	D.	E.	F.
1. Page offset	14	15	16	19	20	Other
2. VPN	14	15	16	19	20	Other
3. PPN	14	15	16	19	20	Other

Agenda

- Virtual Memory and Virtual Addresses
- Paged Memory and Address Translations
- Practice
- **Page Table Details**
- Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- Caches vs Virtual Memory

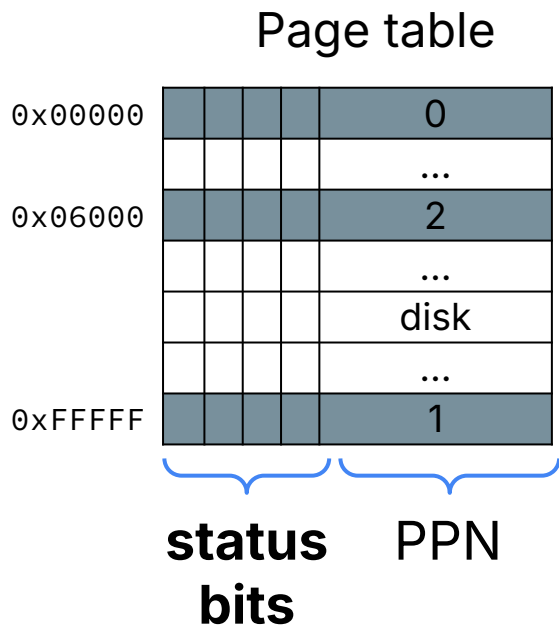
What does a Page Table look like?

- One page table per process.
 - One entry per virtual page number.
 - Entry has **physical page number** (if in memory) as well as **status bits** (more later).
- A page table is NOT a cache
 - A page table is a lookup table!
 - It does NOT have data.
 - All VPNs have an entry in the page table.
 - Index 0 -> stores which physical page corresponds to VPN 0



What does a Page Table Entry look like?

- Each page table entry stores:
 - The physical page number associated with the given VPN
 - Status bits/metadata
- For this class, we'll talk about the following:
 1. Valid Bit
 2. Dirty bit
 3. Write protection bit



Valid Bit

- Page table entries track a **valid bit** to indicate if the page is in memory (DRAM) or only on disk (similar to valid bit in a cache).
 - On each memory access, first check if page table entry is “valid.”
- Valid/on → In main memory
 - Read/write data as directed by process.
- Not Valid/off → On disk
 - Trigger **page fault** exception
 - OS intervenes to allocate the page into DRAM
 - If out of memory, first select a page to replace in DRAM
 - Store outgoing page to disk.
 - Read requested page from disk into DRAM.
 - Finally, read/write data as directed by process.

Dirty Bit

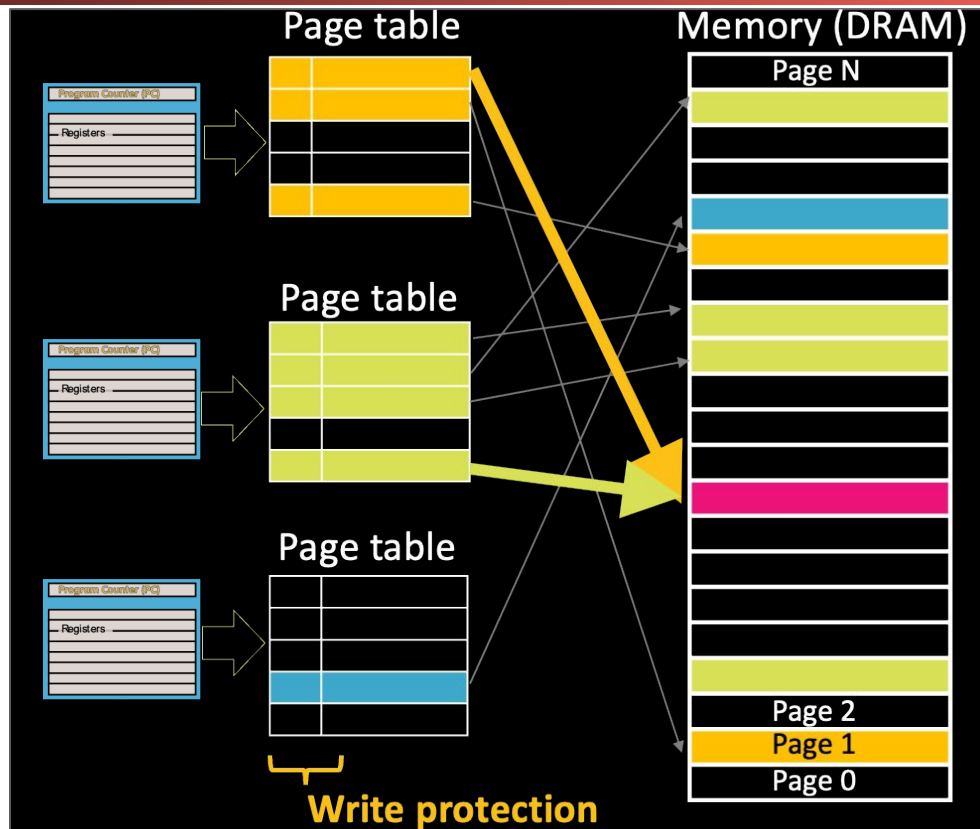
- Main memory acts like a “cache” for secondary memory/disk.
 - Should writes always go directly to disk (write-through), or
 - Should writes only go to disk when page is evicted (write-back)?
- All virtual memory systems use **write-back**.
 - Disk accesses take way too long!
- Recall that write-back policy requires a **dirty bit** to keep track of pages that have an unsynced write.
 - When a page gets replaced:
 - Dirty bit on: Write outgoing page back to disk.
 - Dirty bit off: No disk write.

Write Protection Bit

- Page table **write protection bit**: on = page is “protected.”
 - Writing to a protected page triggers an OS exception.
- Useful for pages that should never be modified
 - Ex. Program code, system data, etc.
- The OS acts to **protect** programs:
 - Isolate memory between processes.
 - Each process gets dedicated “private” memory.
 - Errors in one program won’t corrupt memory of other programs.
 - Prevent user programs from messing with OS’s memory.
 - Allow memory sharing where safe.

Write Protection bit use case: Safe Memory Sharing

- Each process gets its own page table
 - Normally, different processes have different data
- What if two processes want to share the same data?
 - Have their page table map two virtual pages to the same physical page!
- Common use case: two processes accessing the same dynamically-linked library (like `stdlib.h`)
- Don't want to let programs modify `stdlib`, so turn on write protection for those pages.



How big is a page table?

We have a system with:

- 32-bit virtual addresses
- 4 KiB pages
- 32-bit page table entries

How much memory does our page table take? How many pages?

How big is a page table?

We have a system with:

- 32-bit virtual addresses
- 4 KiB pages
- 32-bit page table entries

How much memory does our page table take?

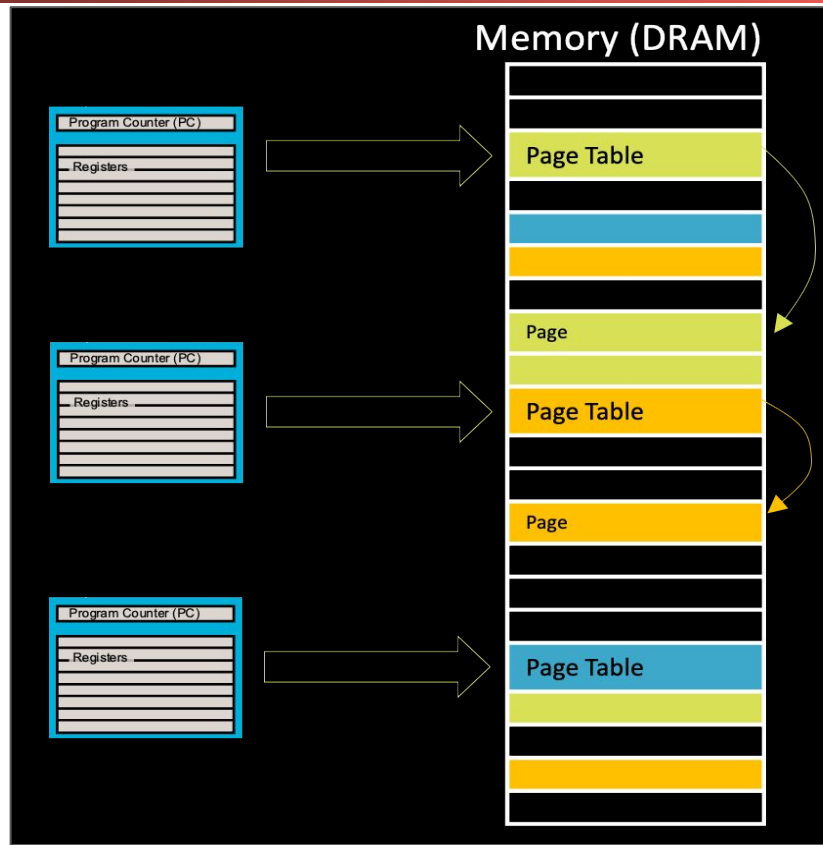
- 2^{12} bytes per page
- 32-bit virtual address $\rightarrow$ 20 bit VPN, 12 bit page offset
- 2^{20} PTEs = 2^{22} bytes = 4 MiB

How many pages?

- 4 MiB / 4 KiB = 1024 pages

Page Tables are Stored in Memory

- Each page table is ~4 MiB
 - Not bad compared to main memory (often on the order of GiBs), but too big for a cache.
 - Can be improved with two layer page table (see skipped slides)
- Solution: store page tables in main memory.
- **Caveat:** Two (slow) memory accesses per lw/sw on cache miss!
 - Read **page table** (stored in main memory) to translate to physical address
 - Read **physical page**, also in main memory
- To minimize performance penalty:
 - Transfer **cache lines** (not words) between DRAM and processor cache
 - Use a cache for frequently used page table entries ... (TLB)



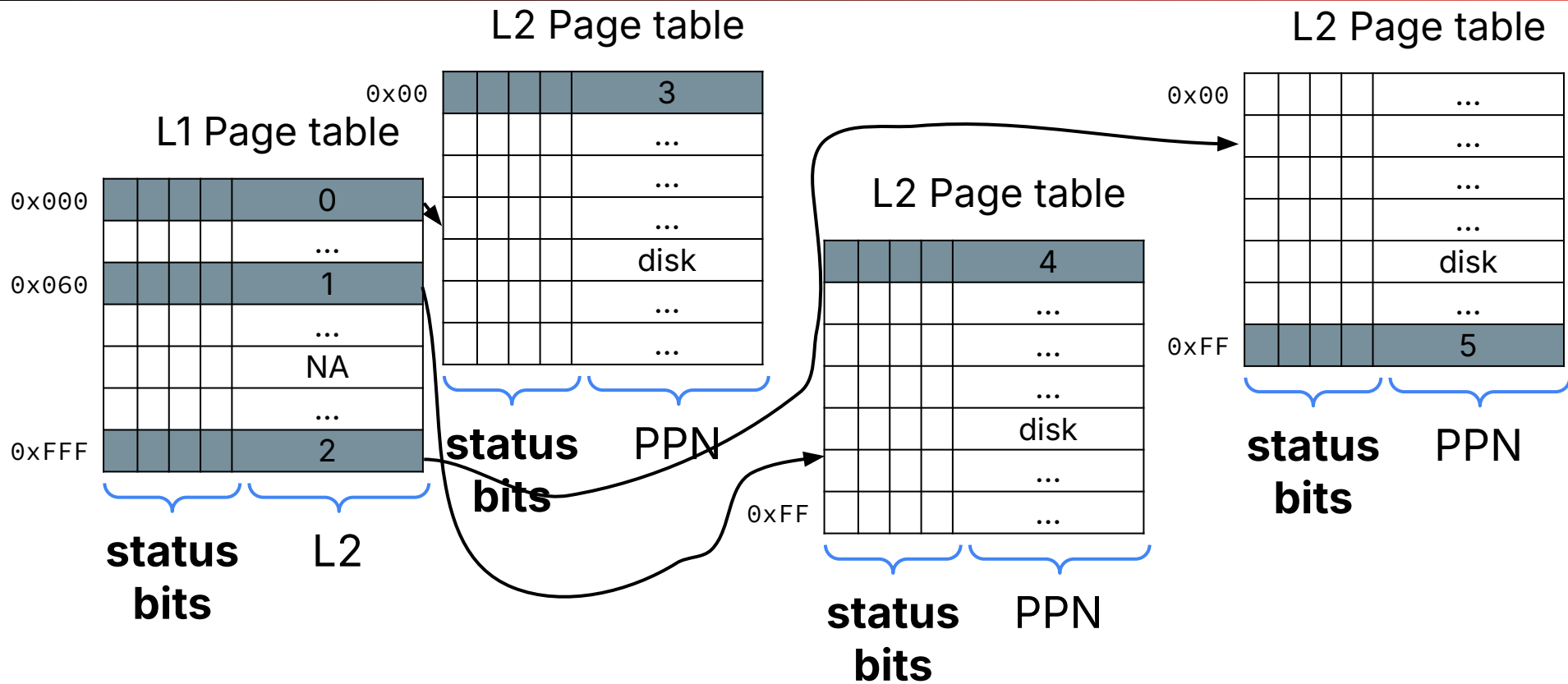
4 MiB is still kind of big

- Our page table is currently 4 MiB
 - Not bad compared to main memory (often on the order of GiBs), but it's still big
- We use a page table because most memory in a program is empty
- Interesting note: Most VPNs in a page table are empty as well...
- Solution: Make a page table of page tables!
 - Called a multi-level page table

Multi-level page tables

- Our page table starts at an L1 page table
 - Each entry in the L1 page table corresponds to a **portion** of the original (big) page table
 - Corresponds to the **top** bits of the VPN
- If **any** entry in that part of the page table is valid, allocate an L2 page table, and note its location (physical page) in the L1 page table
- The L2 page table acts as a regular page table and stores PPNs of the corresponding virtual page
 - Slot in L2 page table corresponds to **bottom** bits of the VPN
- For optimal size, VPN should be split evenly between L1 and L2 page table

Multi-level page tables



How big is each L1/L2 page table?

We have a system with:

- 32-bit virtual addresses
- 4 KiB pages
- 32-bit page table entries
- VPN split in half between L1 and L2 tables

How much memory does each L1/L2 page table take? How many pages?

How big is a page table?

We have a system with:

- 32-bit virtual addresses
- 4 KiB pages
- 32-bit page table entries
- VPN split in half between L1 and L2 tables

How much memory does our page table take?

- 2^{12} bytes per page
- 32-bit virtual address $\rightarrow$ 20 bit VPN, 12 bit page offset $\rightarrow$ 10 VPN bits per table
- 2^{10} PTEs = 2^{12} bytes = 4 KiB = 1 page

The "Magic" Page Table

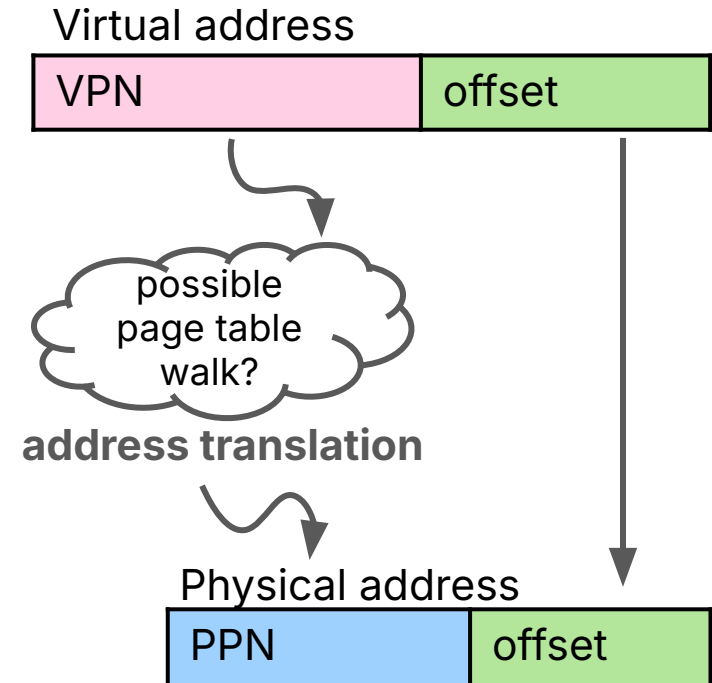
- If we use a 2-layer page table with a 10-10-12 split (L1-L2-Page offset), and PTEs are 32 bits long, then **every page table takes up exactly one page of memory**
- For 32-bit systems, this is really efficient
 - So page sizes of 4 KiB became fairly common
- Previous semesters tended to use this particular example for multi-level page tables (which were in scope at that time)
- Currently out of scope, but interesting to know.

Agenda

- Virtual Memory and Virtual Addresses
- Paged Memory and Address Translations
- Practice
- Page Table Details
- **Translation Lookaside Buffer (TLB)**
- Address Translation: Best/Worst Case
- Caches vs Virtual Memory

Address Translation: Avoid Page Table Walks

- Good Virtual Memory design should be fast (~1 clock cycle) and space efficient.
 - However, every instruction/data access needs **address translation**.
- A **page table walk** is the process of accessing the page table in memory.
 - If page tables are in memory, then each memory access takes several memory accesses to complete (very slow)
- Solution: Cache some address translations in the **Translation Lookaside Buffer (TLB)**.



The TLB Caches Address Lookups

- A page table stores VPN-PPN mappings
- The **translation lookaside buffer (TLB)** is a cache of VPN-PPN mappings.
 - The TLB is much closer to CPU and caches.
 - Predates data caches (so uses an older name)
- Unlike a data cache, does NOT store the data in the page
 - Just speeds up the process of looking for the PPN in the page table, still needs to access the physical page to get actual data

VPN	PPN
0x00004	0x60C25E6
0x00005	0x71DB139
0x00009	0x45099CD

The TLB

VPN	PPN
...	...
0x00004	0x60C25E6
0x00005	0x71DB139
0x00006	0xEC70DB7
0x00007	0xAB12BF4
0x00008	0x2158D55
0x00009	0x45099CD
...	...

Process 1
Page Table

TLB Details

- Technical details:
 - Doesn't store much data, so relatively small
 - Fully associative (usually)
 - Replacement policy: FIFO (stores the most recently accessed pages)
- Can be multi-level, but we'll restrict to single-level TLB for this class
- TLB is on the core, but needs care when the core switches active process
 - Each process has different page table, so different VPN-PPN mapping
 - Easiest: **OS flushes** all/part of the TLB by invalidating its entries.
 - Harder: track which process corresponds to which entries

VPN	PPN
0x00004	0x60C25E6
0x00005	0x71DB139
0x00009	0x45099CD

The TLB

VPN	PPN
...	...
0x00004	0x60C25E6
0x00005	0x71DB139
0x00006	0xEC70DB7
0x00007	0xAB12BF4
0x00008	0x2158D55
0x00009	0x45099CD
...	...

Process 1
Page Table

Agenda

- Virtual Memory and Virtual Addresses
- Paged Memory and Address Translations
- Practice
- Page Table Details
- Translation Lookaside Buffer (TLB)
- **Address Translation: Best/Worst Case**
- Caches vs Virtual Memory

Address Translation Example

- Different cases of **address translation** impact the **end-to-end latency** of a single memory access.
 - Memory access = address translation + data access

"I want to read
0x00004ABC."

orange

firefox

TLB

intellij

CPU

Page Table 1.

Page Table 2.

Page Tables
Each entry 4 bytes.

...banana...

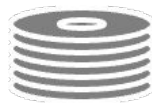
...orange...
(page with data)

...apple...

...

Main Memory
Each page 0x1000 bytes.

Disk



[Case 1, Best] TLB Hit

Address Translation: Instant (~1 clock cycle)

"I want to read
0x00004ABC."

VPN 4 in TLB!



firefox

```
lw    s5 12(a2)
srli  t2 t0 3
...
```

intellij

```
addi  s2 x0 3
jal   label
...
```

CPU

VPN	PPN
0x00004	0x8C121D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

VPN 3

VPN 4

VPN 5

...
0xAB12BF5
0x82C121D
0xD01A3F1
...

VPN 3

VPN 4

VPN 5

...
0xAB12BF4
disk
0x2158D55
...

Page Tables

Each entry 4 bytes.

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory

Each page 0x1000 bytes.

Disk

...
...
...
beans
orange
...
...
...

[Case 2, Worse] TLB Miss + Page Table Walk

Page table walk (~100 cycles): Go to main memory, read page table.

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.



firefox

```
lw    s5 12(a2)
srli  t2 t0 3
...
```

intellij

```
addi  s2 x0 3
jal   label
...
```

CPU

VPN	PPN
0x000002	0x30219D
0x000005	0x2158D55
0x000009	0x45099CD

TLB

**VPN 4 in
Page Table!**



...
VPN 3 0xAB12BF5
VPN 4 0x82C121D
VPN 5 0xD01A3F1
...

...
VPN 3 0xAB12BF4
VPN 4 disk
VPN 5 0x2158D55
...

Page Tables
Each entry 4 bytes.

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory
Each page 0x1000 bytes.

Disk

...
...
...
beans
orange
...
...
...

[Case 2, Worse] TLB Miss + Page Table Walk, cont.

Page table walk (~100 cycles): Go to main memory, read page table.

Update TLB so we have this VPN/PPN mapping handy for next time.

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.

Update TLB.

firefox

```
lw    s5 12(a2)
srli  t2 t0 3
...
```

intellij

```
addi  s2 x0 3
jal   label
...
```

CPU

VPN	PPN
0x00004	0x8C121D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

**VPN 4 in
Page Table!**



VPN 3	0xAB12BF5
VPN 4	0x82C121D
VPN 5	0xD01A3F1
	...

	...
VPN 3	0xAB12BF4
VPN 4	disk
VPN 5	0x2158D55
	...

Page Tables

Each entry 4 bytes.

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory

Each page 0x1000 bytes.

Disk

...
...
...
beans
orange
...
...
...

[Case 3, Worst] Page Fault

Page fault. Go to disk to load page (~1000s cycles)

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.



firefox

```
lw    s5 12(a2)
srli  t2 t0 3
...
```

intellij

```
addi  s2 x0 3
jal   label
...
```

CPU

VPN	PPN
0x000002	0x30219D
0x000005	0x2158D55
0x000009	0x45099CD

TLB

**VPN 4 has
no PPN in
Page Table.**



...
VPN 3 0xAB12BF5
VPN 4 disk
VPN 5 0xD01A3F1
...

...
VPN 3 0xAB12BF4
VPN 4 disk
VPN 5 0x2158D55
...

Page Tables
Each entry 4 bytes.

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory
Each page 0x1000 bytes.

**Load page
from disk.**

Disk

...
...
...
beans
orange
...
...
...

[Case 3, Worst] Page Fault

Update page table with PPN of the newly-loaded page.

Update TLB so we have this VPN/PPN mapping handy for next time.

"I want to read
0x00004ABC."

VPN 4 NOT in TLB.

Update TLB

VPN	PPN
0x00004	0x8C121D
0x00005	0x2158D55
0x00009	0x45099CD

TLB

**VPN 4 has
no PPN in
Page Table.**

...	
VPN 3	0xAB12BF5
VPN 4	0x82C121D
VPN 5	0xD01A3F1
...	

Update page table.

	...
VPN 3	0xAB12BF4
VPN 4	disk
VPN 5	0x2158D55
	...

Page Tables
Each entry 4 bytes.

...	...
banana	0xD01A3F1000
...	...
apple	0xAB12BF5000
potato	0xAB12BF4000
...	...
orange	0x82C121D000
...	...
carrot	0x2158D55000
...	...
Page Table	0x120331D000
Page Table	0x120331C000

Main Memory
Each page 0x1000 bytes.

**Load page
from disk.**

Disk
...
...
...
beans
orange
...
...
...

firefox

```
lw    s5 12(a2)
srli  t2 t0 3
...
```

intellij

```
addi  s2 x0 3
jal   label
...
```

CPU

Summary: TLB Hits, Page Table Walk, Page Fault...

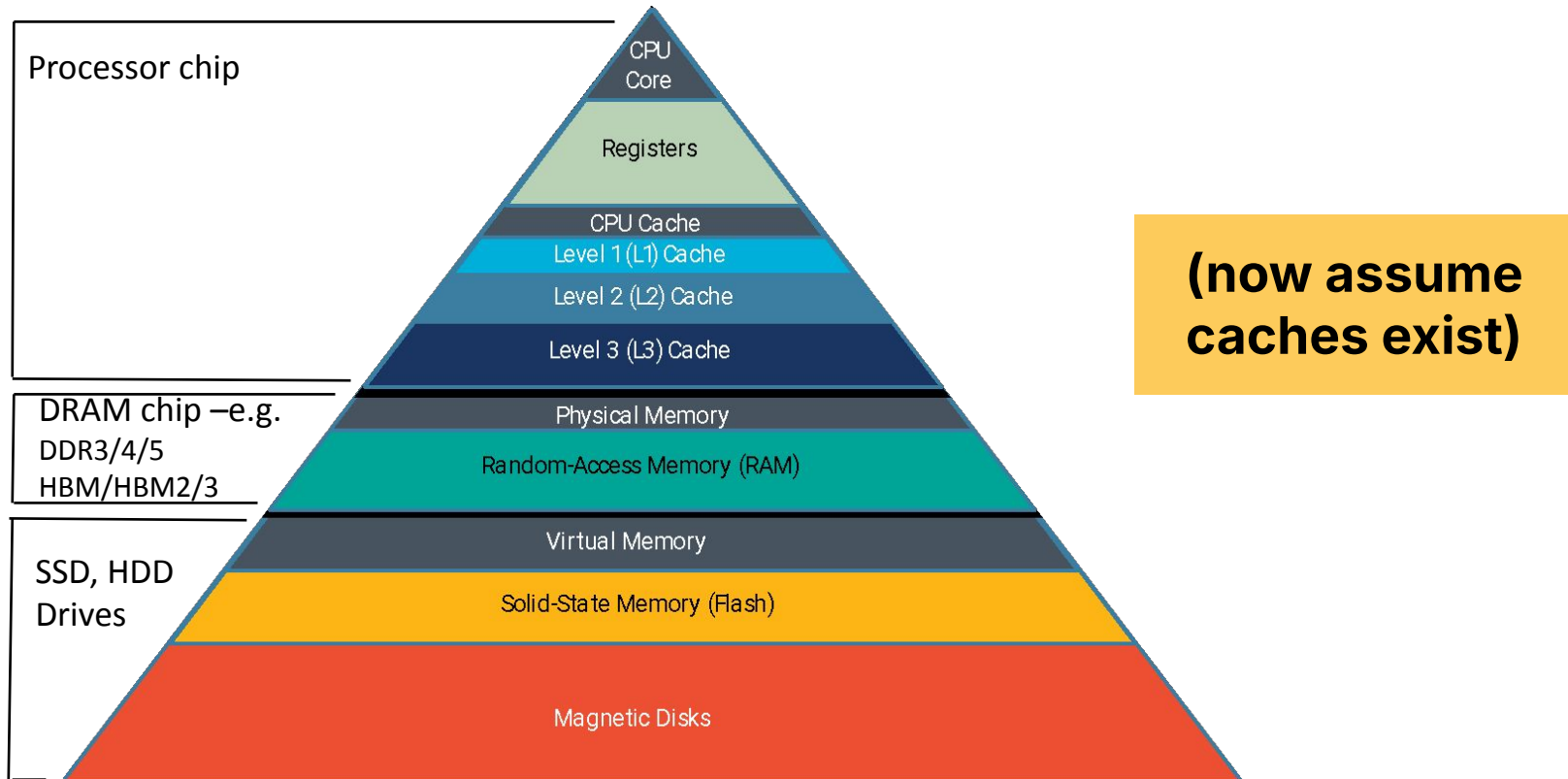
- Three cases for an address translation:

	TLB	Page Table Walk	Disk access
Best	Hit 	Not visited	Not visited
Worse	Miss 	Page Table Entry Valid 	Not visited
Worst	Miss 	Page Fault 	Load page 

Agenda

- Virtual Memory and Virtual Addresses
- Paged Memory and Address Translations
- Practice
- Page Table Details
- Translation Lookaside Buffer (TLB)
- Address Translation: Best/Worst Case
- **Caches vs Virtual Memory**

The Entire Modern Memory Hierarchy



Another View of Memory Hierarchy

L1 Cache

V	Tag	Data
		Line (64B)
...	...	...

L2 Cache

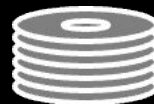
V	Tag	Data
		Line (128B)
...	...	...

DRAM (primary memory)

Data Page
(4KiB)



Disk (secondary memory)



Data Page
(4KiB)



Memory Units

- Lines, pages, bytes, words are all units in the memory hierarchy.
 - Cache lines ~ 64 B or 128 B
 - Pages: ~4 KiB

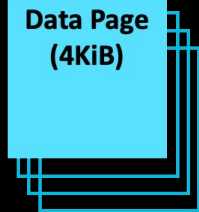
L1 Cache

V	Tag	Data
		Line (64B)
...	...	...

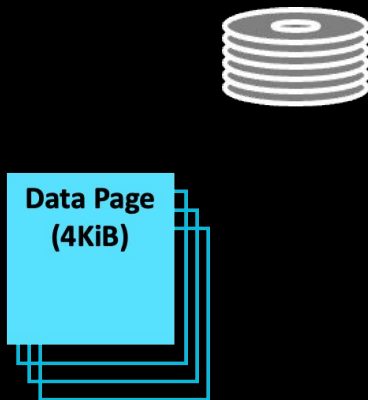
L2 Cache

V	Tag	Data
		Line (128B)
...	...	...

DRAM (primary memory)



Disk (secondary memory)



Caches have copies of data

- “Cache” Paradigm: Data at each level is a quick-access copy of data at a lower level in the memory hierarchy.
- Similarly, data pages in main memory are “cached” versions of pages on disk.

L1 Cache

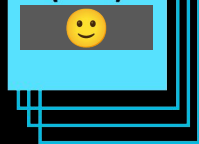
V	Tag	Data
		☺
...	...	...

L2 Cache

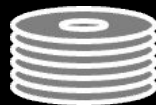
V	Tag	Data
		☺
...	...	...

DRAM (primary memory)

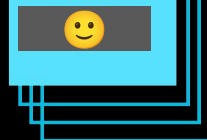
Data Page
(4KiB)



Disk (secondary memory)

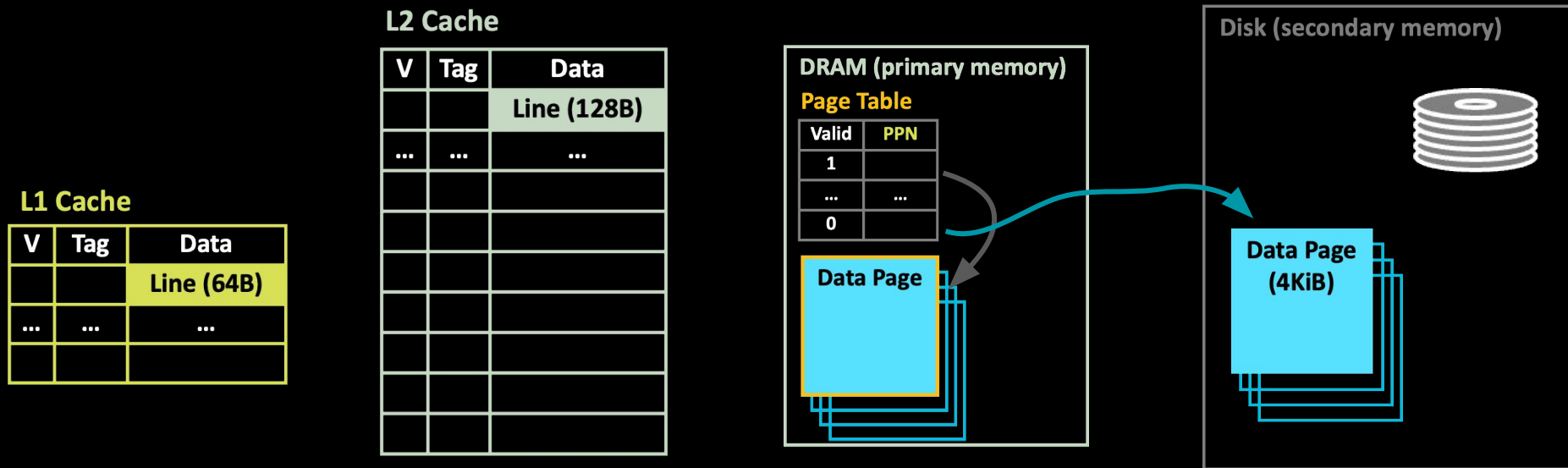


Data Page
(4KiB)



Page Tables don't copy data

- A Page Table translates addresses.
- Page tables facilitate **demand paging**.
 - Page Table keeps track of page status/location only (not data).

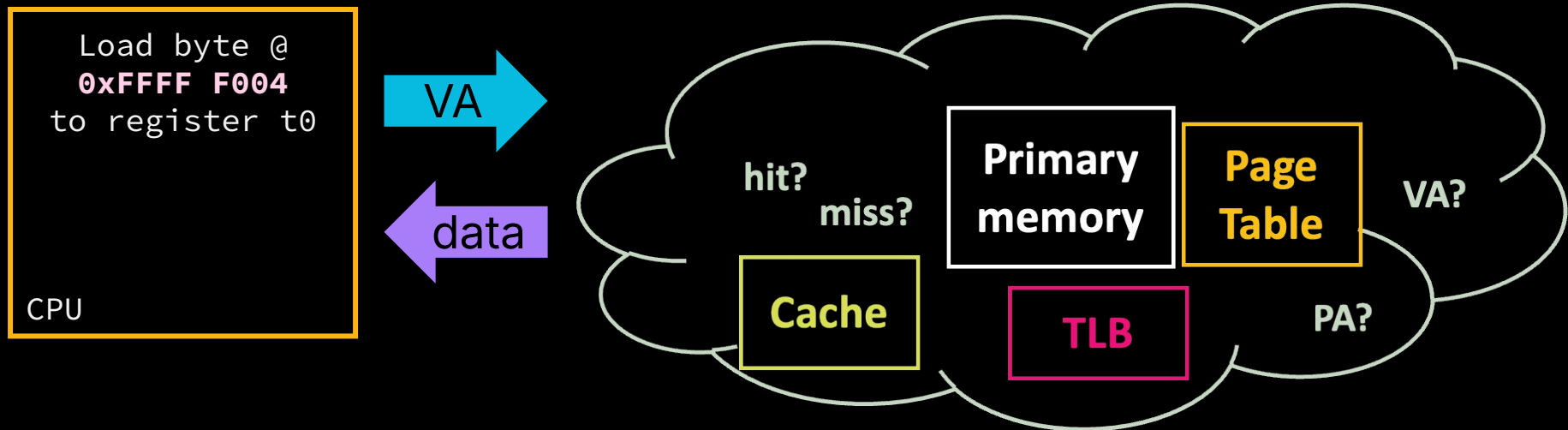


Caches vs. Virtual Memory

	Caches	Virtual Memory
In memory hierarchy	Caches ↔ Memory	Memory ↔ Disk
Memory unit	Block (~64 bytes)	Page (~4096 bytes)
Miss	Cache Miss	Page Fault
Associativity	Direct-mapped, N-way set associative, fully associative	Fully associative (pages can go anywhere in memory)
Replacement policy	Least-recently-used (LRU) or random	LRU (most common), FIFO, or random
Write policy	Write-through or write-back	Write-back

Putting it All Together: PIPT

- Can a cache hold the requested data if the corresponding page is not in main memory?
- On a memory reference, what should we access first?
- When should we translate virtual addresses?



Putting it All Together: PIPT

- Other more complicated designs exist, but in this class we assume **physically indexed, physically tagged (PIPT) caches**.
 - 1. First translate **virtual address** to physical address.
 - 2. Then use the **physical address** to access the data cache.

